STATISTICS & MACHINE LEARNING · LESSON 30 OF 30

# Regression Metrics: MAE, MSE, RMSE and R-Squared

Measure how wrong a numeric prediction is, in the units the business uses. Compute MAE, MSE, RMSE and R-squared on a delivery-time model, learn which metric punishes big mistakes, see why R-squared alone misleads, and explain each metric to a manager.

Gap-fill lesson | ✦ 2026 | Intermediate | ⏱ 100 min

**By the end of this lesson you can**

- Compute MAE, MSE, RMSE and R-squared by hand and with scikit-learn
- Explain which metric matches a business cost of errors
- Compare a model with a simple baseline
- Explain why R-squared can mislead with flexible models

**Datasets used**

`delivery_times.csv`

## Why the error metric matters

A forecasting or pricing model gives a number, and someone must decide whether it is good enough. The answer depends on the **cost of being wrong**. Being ten minutes late on one delivery may matter less than one delivery that is three hours late. Each regression metric measures error in a different way, and choosing the right one is a business decision as much as a statistical one.

CASE STUDY **How wrong are the delivery estimates?**

A logistics team gives customers a delivery time estimate. The model predicts minutes; the actual times are in the delivery log. The operations head asks: "On a typical delivery, how far off are we, in minutes?" and "How bad are the very late ones?" Different metrics answer each question.

## Step 1: the errors

For each delivery, the **error** (residual) is actual minus predicted. Positive means the delivery took longer than predicted. Every metric below summarises these errors.

Python

```python
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

d = pd.read_csv("delivery_times.csv")
X = pd.get_dummies(d[["city"]], drop_first=True).astype(float)   # city as 0/1 (Pune is the baseline)
y = d["delivery_minutes"].values

model = LinearRegression().fit(X, y)
pred = model.predict(X)
err = y - pred
print(pd.Series(err).describe().round(2))
```

OUTPUT

```
count    115.00
mean      -0.00
std        6.96
min      -16.60
25%       -4.91
50%       -0.80
75%        4.75
max       17.09
dtype: float64
```

The model here uses only the city, so it is a deliberately simple model. The metrics are the same for any regression, however complex.

## Step 2: MAE, the average absolute error

**Mean absolute error (MAE)** is the average of the absolute errors, in minutes. It answers "on a typical delivery, how far off are we?" Every error counts in proportion to its size.

Python

```python
mae = np.mean(np.abs(err))
print("MAE (minutes):", round(mae, 2))
```

OUTPUT

```
MAE (minutes): 5.54
```

MAE is easy to explain and is **robust to outliers**: one extreme delivery moves it only by its own size divided by the number of deliveries.

## Step 3: MSE and RMSE, which punish large errors

**Mean squared error (MSE)** squares each error before averaging, so one 60-minute miss counts four times as much as a 30-minute miss in the same total. **Root mean squared error (RMSE)** takes the square root of MSE, which returns the result to the original units (minutes).

Python

```python
mse = np.mean(err ** 2)
rmse = np.sqrt(mse)
print("MSE (minutes squared):", round(mse, 2))
print("RMSE (minutes):", round(rmse, 2))
```

OUTPUT

```
MSE (minutes squared): 48.08
RMSE (minutes): 6.93
```

RMSE is always at least as large as MAE. The bigger the gap between them, the more the errors are dominated by a few large misses. Compare them on the same data:

| Metric | Value | What it tells you |
|---|---|---|
| MAE | about 5–6 minutes | The typical miss |
| RMSE | larger than MAE | The typical miss with large misses weighted more |
| Gap RMSE − MAE | small | Errors are fairly even |

## Step 4: R-squared, the share of variation explained

**R-squared** compares the model's errors with the errors of a model that always predicts the average. A value of 0.6 would mean the model explains 60% of the variation in delivery times; the model in this lesson reaches only about 0.02 (see Step 5). A value of 0 means it is no better than the average; below 0, it is worse.

Python

```
from sklearn.metrics import r2_score
r2 = 1 - np.sum(err ** 2) / np.sum((y - y.mean()) ** 2)
print("R-squared (by hand):", round(r2, 3))
print("R-squared (sklearn):", round(r2_score(y, pred), 3))
```

OUTPUT

```
R-squared (by hand): 0.023
R-squared (sklearn): 0.023
```

Both lines agree, which is the check that the formula is correct.

## Step 5: check against a baseline

A metric means little without a comparison. The simplest baseline always predicts the average delivery time. Your model must beat it.

Python

```
from sklearn.metrics import mean_absolute_error, root_mean_squared_error

baseline = np.full_like(y, y.mean(), dtype=float)
print("baseline MAE:", round(mean_absolute_error(y, baseline), 2),
      " model MAE:", round(mean_absolute_error(y, pred), 2))
print("baseline RMSE:", round(root_mean_squared_error(y, baseline), 2),
      " model RMSE:", round(root_mean_squared_error(y, pred), 2))
```

OUTPUT

```
baseline MAE: 5.68  model MAE: 5.54
baseline RMSE: 7.01  model RMSE: 6.93
```

In this data the city-only model barely beats the baseline: its MAE is about 5.5 minutes against about 5.7 for the average, and its R-squared is about 0.02, so city explains almost none of the variation in delivery time. That is an honest and useful result: a more complex model is unlikely to help unless you add inputs that carry real information, such as distance, traffic or courier.

## Step 6: when R-squared misleads

R-squared never gets worse when you add more inputs to a model, even useless ones, and very flexible models can fit the training data closely while predicting new data poorly. Always check error metrics on **new** data, not the training data, and never judge a model by R-squared alone.

Python

```
rng = np.random.default_rng(0)
noise = pd.DataFrame(rng.normal(size=(len(d), 40)), columns=[f"noise_{i}" for i in
range(40)])
X_noisy = pd.concat([X.reset_index(drop=True), noise], axis=1)
pred_noisy = LinearRegression().fit(X_noisy, y).predict(X_noisy)
print("R-squared with 40 random columns added:", round(r2_score(y, pred_noisy), 3))
print("Original R-squared:", round(r2_score(y, pred), 3))
```

OUTPUT

```
R-squared with 40 random columns added: 0.417
Original R-squared: 0.023
```

Random columns carry no information, yet R-squared rises. The fix is to measure on held-out data and use adjusted R-squared or a validation score for model comparison.

::::warn R-squared is not accuracy R-squared says how much variation is explained, not how close the predictions are. A model can have an R-squared of 0.9 and still be off by an unacceptable number of minutes. Report an error metric in business units beside it. ::::

## Choosing a metric for the business

| Business cost | Metric to lead with | Why |
|---|---|---|
| Every minute of error costs the same | MAE | Simple, robust, in minutes |
| One large miss is much worse than many small ones | RMSE | Large errors weigh more |
| Comparing models on one dataset | RMSE or MAE, plus baseline | Same units, easy to rank |
| Explaining how much the inputs matter | R-squared, on held-out data | Share of variation explained |
| Money from errors in pricing | MAE or RMSE in currency | Convert to rupees for the manager |

INTERVIEW CORNER

### When would you prefer MAE over RMSE?

When all errors matter in proportion, or when the data has outliers that you do not want to dominate the score. Use RMSE when large errors are disproportionately costly, such as a shipment that is hours late. Report both when it is unclear, and say which one the business decision depends on.

::::key What to remember - MAE is the typical absolute error in the business's units and is robust to outliers. - MSE squares errors; RMSE returns to the original units and punishes large misses. - R-squared is the share of variation explained, and it can mislead with flexible models. - Always compare against a baseline (predict the average) and measure on held-out data. - Pick the metric from the cost of error, and explain it in the unit the business uses. ::::

## Quick quiz

5 questions · tap an answer to check it

**1. A model's MAE is 4 minutes and its RMSE is 9 minutes. What does this suggest?**

A All errors are about 9 minutes

B A few large errors are pulling RMSE up above the typical error

C The model is perfect

D The units are wrong

**Answer: B.** RMSE is at least MAE, and a big gap means large errors dominate.

**2. Which metric returns to the original units while still penalising large errors?**

A MSE

B RMSE

C R-squared

D MAE

**Answer: B.** MSE is in squared units; its square root (RMSE) is in the original units.

**3.** Adding 40 random columns raised R-squared. What is the lesson?

A More inputs always improve a model

B R-squared alone can rise without real improvement, so use held-out data

C Random columns are useful

D R-squared is always wrong

**Answer: B.** The metric rewards added inputs even when they carry no information.

**4.** What is the most useful first comparison for a new regression model?

A Its R-squared on the training data

B Its error against a simple baseline that predicts the average

C Its number of parameters

D Its training time

**Answer: B.** A model must beat a simple baseline to be worth using.

**5.** An R-squared of 0.8 means:

A The model is 80% accurate

B The model explains 80% of the variation in the target

C Errors are 80% of the mean

D 80% of predictions are correct

**Answer: B.** R-squared is the share of variation explained.

EXERCISE **Explain the metrics to a manager**

Using `delivery_times.csv`, fit a model with city and with one extra feature you invent (for example, a random "weekday" flag, or any column you can create without leaking the target). Report MAE, RMSE and R-squared for the baseline and both models in one table, and write three sentences a manager can act on.

**Show solution (worked answer)**

Hints: create the extra feature only from information available before delivery. Use the same metric functions for all models. Report the RMSE-to-MAE gap as a sentence: "most deliveries are within X minutes; a few are much later."

## Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.